Hate Speech Detection and Classification Analysis

Aby Babu Alappat
Course: Information Security
Student Id: J01210514

Section 1

5 KAGGLE NOTEBOOKS FOR MILESTONE 2

Code 1: Cyberbullying Prediction Notebook

The first thing I saw was someone else using the same data that I was using. It is the HateXplain cyberbullying dataset from Kaggle. This data collection contains comments labeled as well nice or very undesirable. What I found interesting was how they organized their data. The data system changed to reflect diverse types by showing race, sexuality and religion.

This notebook only utilizes simple preprocessing techniques to clean up the data. With their information ready to use, they turned their attention to setting up their capabilities. To be able to make a machine learning model ready, they used TF-IDF to convert the text into numbers. This project will be similar to mine so I want to see how they did it.

```
df = pd.read_csv('/kaggle/input/cyber-bullying-data-for-multi-label-classification/final_hateXplain.csv')
print('Full train dataset shape is {}'.format(df.shape))
```

Three different models were used: Logistic Regression, CatBoost, and Naive Bayes, for testing. By comparing multiple algorithms to each problem individually it is possible to determine which algorithm works best for that specific problem. They also used an 80/20 split for training and testing.
Their primary aim was achieving a very high F1-score. For a more accurate evaluation of the models, they focused on the F1-score instead of accuracy. However, our data set is unbalanced - the data set is large, but there is significantly more of normal comments than hateful comment.
Compared to my milestone 1 work, I included the entire 60,444 row data set, as well as the annotations, using two different updated and new annotators while they used the controlled 20,109 row data set.
In the first phase of my work. I spent a lot of time focusing on data quality assessment and exploratory analysis. This allowed me to examine things such as missing values, duplicates, and the distribution of characteristics in my data. Their notebook focused quickly on the testing and training stages.
Testing multiple models is one thing I can learn from this notebook . In my milestone 1, I mainly focused on Random Forest. Furthermore, for my research project, I could also explore alternative techniques,

including logistic regression, to analyze results. Examining data preparation after finding great summary statistics also helps me feel a bit more secure.

Code 2: ML-NLP Supervised Cyberbullying Detection

The clean layout of the secondary notebook blew me away. The author approaches building the discrimination system in a more professional and businesslike fashion than in the previous task. Replace the original with 20,109 rows of post texts that have no multiple annotator versions.
Their text preprocessing workflow really caught my attention immediately. Before running any software, people carefully clean the text to operate. In order to clean the text, the programmers removed unwanted words like "the", "is", and "and", as well as performed a lemmatization process that narrowed words down to their simplest form, and finally, the programmers made sure to remove any special characters from the text. Here's how they did it.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['lemm_text'])
```

I think these actions could make my project better too.
For milestone 1 I applied standard cleaning methods like taking care of missing values and removing duplicates however I didn't handle text and other preprocessing necessities. This could allow my new model to be more accurate.
The author’s (code 2) thought of the use of sklearn Pipeline in the notebook is intriguing. The code 2 chain all the steps together into one process, the text would of course have been preprocessed with TF-IDF and the categorical features with one-hot encoding. My current way of doing things is less clean and not as effective compared to this code 2 .

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([
   ('preprocessing', ColumnTransformer([
      ('tfidf', TfidfVectorizer(max_features=1000), 'post_tokens'),
      ('onehot', OneHotEncoder(), ['target'])
   ])),
   ('classifier', RandomForestClassifier(n_estimators=100))
])
```

The Random Forest classifier is the most used. Confirming my method of model selection strengthens my belief. Based on the data this text was worked from, the original author(code 2) probably got either 80% accuracy or 85% with the use of text classification with Random Forest. This is just their guess.
Similar to my paper, two different techniques have several advantages each. It’s(code 2) pipeline is more elegant and production-ready. Before I started any sort of modeling, it was more important to me that I

understand the data inside and out which ended up being time consuming. I plan on incorporating the most effective aspects of analysis and pipeline when working on milestone 2.

Code 3: Cyberbullying Classification with Deep Learning.

This notebook has a new approach to innovation than the other two. The approach is gung-ho deep learning. I was curious to know how advanced methods would compare with my machine learning technique normally. They used a worrying number of observations gleaned from 20,109 messages. The preprocessing here is extensive. They remove unnecessary words, strip punctuation, and divide up text. They accomplish what the previous method was unable to by transforming words into long stretches of digits that the neural network is prepared to understand.

```
def preprocess_text(text):
    text = text.lower()
    text = re.sub(string.punctuation, '', text)
    tokens = text.split()
    stopwords_set = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stopwords_set]
    return ' '.join(tokens)
```

Their transformer-based BERT system is cutting-edge for analyzing and understanding language. The code develops a complicated model. This model contains multiple layers and "especially attention" features.

```
from transformers import TFBertModel, BertTokenizer
bert_model = TFBertModel.from_pretrained('bert-base-uncased')
```

The crucial problem that surprised me concerns an overfitting issue they faced. The model's high accuracy rate appears out for a little bit but right after, falters fail because the accuracy drops to 54%. Even extremely sophisticated models of deep learning fail when they over rely and memorize information from their training data.

Epoch 1/75: accuracy: 0.8637 - val_accuracy: 0.5009
Epoch 5/75: accuracy: 0.8891 - val_accuracy: 0.5668

The programmers employed high tech resources to achieve these higher goals despite the huge resource dependencies. With the Random Forest model results being more consistent here isn't a bigger need for a better or more expensive computer . That makes me feel assured in the path I've selected. The BERT model is extremely advanced but somewhat off target . For my second milestone, I intend to keep working with Random Forest, modified with effective feature sets(PCA, LDA, Chi-Square). I think this will give me better, more reliable results than trying to use complex, deep learning tools, which I do not fully

understand and don’t have any way to debug.

Code 4: Cyberbullying Classification Using BERT (93% Accuracy)

The fourth notebook is impressive because BERT actually works right and they scored 93% test accuracy. Whereas previous models heavily overfitted, this current model gives the author of a much more effective one. They used the full 60,444 entries then later reduced it to an aggregate of 20,148 individual posts.
Their data processing was intelligent. They combined many annotators and used their votes to ultimately create a label. The annotators basically agree to let 2 judgements out of 3 define an offensive statement by them.

```
grouped_df = hate_df.groupby('post_id')['label']
unique_label = grouped_df.agg(lambda x: x.value_counts().index[0])
```

The text preprocessing here is comprehensive. They remove irrelevant words and numerical information with marks. Their software also handles emojis which are necessary in online communication.

```
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [word for word in tokens if word not in stop_words]
    return ' '.join(filtered_tokens)
```

It’s(code 4) variation of BERT differs in how it's established. The code uses PyTorch and many advanced algorithms which ensure data is preprocessed and easily accessible. 10 epochs in the model goes from normal to far superior.

Epoch 1: Loss: 0.8238, Accuracy: 62.5%
Epoch 5: Loss: 0.2850, Accuracy: 89.6%
Epoch 10: Loss: 0.0566, Accuracy: 98.1%

93.05 percent accuracy is a fantastic score. The model just barely overfits correctly, and that is far better than the model in the other BERT notebook that had a large gap.
However, I did notice a few downsides. The model's file size is a massive 440 megabytes, which is unusually big. Training took about 30 minutes even with GPU. And they had special hardware that I don't own .

Approximately ninety three per cent determination is great, but my random forest technique with the

proper selection of variables may lead me to eighty to eighty five per cent accuracy and be completed in a shorter time period, according to us. This method does not require expensive equipment. It also shows which component causes each classification to occur. Out of all the other approaches to explain my results, this one works the best.

This notebook demonstrates how to use deep learning correctly . I've learned what it's capable of. If I had unlimited funds and was creating a hate speech detection system i would only use BERT for my project which is more suitable.

Code 5: Hate Speech Classification 93% Accuracy (MLP)

The last Notebook I used has it's own unique approach. Its intermediate balance unlike models but more complex than the restricted BERT. The machine uses a Multi-Layer Perceptron MLP, a form of neural network. The results reached 93% accuracy, comparing well to others results.

The exact dataset is put along with posts giving a total of 20,148 unique posts using majority voting. What specially grabbed my attention was the way they approached feature extraction instead of choosing between CountVectorizer or TDF. Both were applied and combined.

```
cv = CountVectorizer(tokenizer=word_tokenize, ngram_range=(1, 2))
tfidf = TfidfVectorizer(tokenizer=word_tokenize, ngram_range=(1, 2))

def get_features(cv, tfidf, x):
    cv_feature = cv.transform(x['post_tokens'])
    tfidf_feature = tfidf.transform(x['post_tokens'])
    return np.hstack([cv_feature, tfidf_feature]) / 2
```

TF-IDF is better than CountVectorizer because important things are needed to appear more than others. Their combined method gets everything from both of them. Their combined method gets everything from both of them. Bigrams are used to determine the meaning of pairs of words like "hate speech" rather than individual words. Using a better feature could certainly help my project . Right now I'm only using TF-IDF, which might not be as accurate.

The MLP model the code made is smaller than BERT but much larger than random forest. Training occurred for ten cycles with better results as loss went down steadily.

```
mlp = MLPClassifier(random_state=0, batch_size=1000, max_iter=10)
mlp.fit(x_train, y_train.ravel())

# Training output:
# Iteration 1, loss = 1.01212972
```

 # Iteration 5, loss = 0.12397161
 # Iteration 10, loss = 0.03061394

The model reached 93% accuracy with strong performance across all three classes:

- Normal: 96% F1-score
- Offensive: 90% F1-score
- Hatespeech: 93% F1-score

I believed their interactive testing was very impressive. I can type any sentence and it predicts the label.

```
def play(input_text):
    df = pd.DataFrame({'post_tokens': [input_text]})
    combine_feature = get_features(cv, tfidf, df)
    y_pred = mlp.predict(combine_feature)
    return classes[int(y_pred[0])]

play("you are a bad guy")  # Output: 'offensive'
```

The training only took four minutes and it’s a lot faster than BERT. The model didn't become much better, even after 10 cycles, so it needs more. The code could almost hit 100% in accuracy on all of the predictions.

In my opinion, the MLP System is an exciting idea, but I'm still leaning more towards using the alternative option(random forest). Because the Multi Layered Perceptron is "black box", words and features are hard to recognize. This program can allow me to know what certain signs are telling me about hate speech. These PCA, LDA, and Chi-Square feature selection methods operates more smoothly with traditional machine learning methods rather than deep learning. Third, I prefer that my project be kept simple so it isn't too hard to understand. I'm mostly drawing inspiration from their way to combine features for two parts. I should hopefully gain a much better performance by adding bigrams to features in addition to basic TF-IDF. (I might also try multi-layer perceptron as a fourth model.)

Overall Reflection on the Five Notebooks

After completing five different notebooks I’ve learned various methods for detecting cyberbullying. These BERT based systems vary due to possible overfitting from 54% to 93% efficiency rates. These simple approaches like Random Forest likely had 80-85% accurate outputs.

Not all complex models are, by default, a success. The most complex model (BERT in notebook 3) for this task ended up being the most poorly performed due to over fitting. The very best came from either sophisticated deep learning (notebook 4) or moderate complexity models with better feature

engineering (notebook 5).

For my milestone 2, I'm taking several lessons.
1. Text preprocessing is important - I want to add stopword deletion and possibly lemmatization to my workflow.
2. Feature engineering is powerful - Integrating various features and applying bigrams can boost efficiency substantially.
3. Simple can be preferable - My Random Forest method is logical and will probably execute well (80-85% accuracy).
4. clarity has value - Having able to demonstrate which features are significant is important for research work.
5. Multiple models aid - contrasting different strategies provides better understanding than just using one approach.
My way of doing the project is effective and I am confident it will lead to positive results when completed. A fully implemented system would provide higher, if not 93%, accurate ability to differentiate positive comments from negative ones. But for now, my setup can check these patterns and use them, though possibly, to better determine characteristics of hate speech. Thus it can be used to help train future applications.

Section 2

Introduction to Related Work

In beginning my research, the essence of finding successful hate speech detecting processes became the test: The success of methods available. Researchers are near one hundred percent sure that after conducting enough testing anything from complex to simple models might help it. In this summary, I explain and cover the main papers that I used in my research on detecting hate speech and cyberbullying in sources online. I will be comparing my paper to these research papers.I will explain the dataset, modeling techniques, results, and compare the ideas to my own paper.

Paper 1: Detection of Hate Speech using BERT and Hate Speech Word Embedding (2021)

A Google developed model, the deep learning BERT, was coupled with word embeddings designed to analyze unsafe speech. Researchers found that word embeddings often neglect the details of hate language, so they developed new custom empowered embeddings that are trained solely on hate language sets.
The models were trained using a combined variety of hate filled statements. They did two experiments. In the first they used a LSTM with word embeddings and in the second they used transfer learning. The biLSTM model obtained an F1 score of 93 percent and the BERT model has performed better in recording an F1 score of 96 percent. Word embeddings is a very powerful tool in detecting hate speech.

The complexity in this assignment is more challenging to work with and understand. Researchers used deep learning to speed up tedious projects. My first approach of using Random Forest was somewhat

easier and faster to train and easier to understand, using the traditional method of encoding. Their 96 score is definitely better than expected, but the additional 10-15 percent isn't worth the added costs and difficulty that come with completing the project. After this paper, I've decided that textual meaning is crucial. I expect to use the TF-IDF with bigrams for more than just overall meaning, and without deep learning to figure out things like metaphor & anything else really dangerous or really subtle.

Paper 2: Modified TF-IDF with Machine Learning Classifier for Hate Speech Detection (2023)

This paper in the Turkish Journal opened my eyes because it is more likely to happen in my lifetime. The researchers collected hate speech data specifically off the Twitter website. Researchers applied a version of TF-IDF and machine learning models . To improve standard TF-IDF, they included context and specific help to search for hate speech to stop it from spreading globally.

The method that they used was substantial. They decided to do tokenization of all the text. They took out the unnecessary words or the stop words. They tested several machine learning models, including Support Vector Machines, Random Forest, and Logistic Regression. Results from using the Random Forest model with an upgraded TF-IDF showed accuracy tests surpassed others.

Feature engineering is key for achieving the best possible performance with due to meaningful added value. The benefits of the state of the art method validates the plan detailed out in my milestone 2. Their study indicates that using TF-IDF plus methods like Random Forest can probably give a good performance comparable to deep learning. This clearly justifies my approach option. My task is to use the HateXplain dataset to learn to identify normal, hate speech, and offensive. Their work was only on two classes. The people who receive hate speech also give me more details to analyze interests in.

Paper 3: Recurrent Neural Network Based Multiclass Cyberbullying Classification (2024)

Researchers researched cyberbullying detection on Bengali social media which is a tough task for them. A group of researchers collected a dataset, including things that were classified as threats, trolling, sexual harassment and and non-bullying. The approach uses regular neural networks which support capturing sequential patterns. Different algorithms are compared such as svm, a naive bayes, xg-boost, logistic regression, and regular neural networks. The RNN achieved 86% accuracy on their Bengali dataset. The concept of multi-class classification is much more difficult than simple one or the other decisions, distinguishing between similar categories is hard due to the overlapping or confusing factors of each category.

My project also tries to classify 3 types of communications in broader terms: good, offensive, harmful. Because labeling such posts as hate speech can be so complicated, I obtained different annotations from several individuals. As opposed to others that use Bengali, my provided data is based and gathered. They used RNN which is studying the condition of word by word whereas I used TF-IDF that indicates the importance of word. In the first stage of modeling, ensuring my data is accurate was done before shaping models. I believe, getting accurate data is just as crucial as selecting one of best algorithms.

Paper 4: A Comprehensive Framework for Multi-Modal Hate Speech Detection (Nature, 2025)

A research paper appearing in Nature Scientific Reports explains recent success in more effectively detecting hate speech. Reseachers made a system that doesn't just analyze written language, but also video, audio, and pictures. The framework uses CNN for extracting spatial features, LSTM for temporal dependencies, and BERT for text understanding. The authors showed how to identify and combine the most important features of all included data.
Their test ran very well, they almost had a perfect accuracy(98.53% detection accuracy with 97.64% robustness). However, the computational demands are enormous. The system requires GPU groups and significant training time. I think about my project, and this has changed the way I look things over. I'm only using traditional machine learning methods that can run on a regular laptop(google Colab) to detect text-based hate speech.
I can gain insight into the method of feature importance from this given assignment. While I was unable to select high density features, I can determine which words and phrases in hate speech can be detected(chi-square). It is much simpler and theoretically similar to many popular attention mechanisms. These studies concur that an assorted variety of method will allow to exceed one distinct technique if used independently.

Paper 5: Hate Speech Detection and Racial Bias Mitigation in Social Media (PMC, 2020)

This research describes how racial bias exists in hate speech detection systems. The researchers examined files from Twitter that had been marked as racist. The researchers evaluated the models on material that corresponded to either Black American English or white English. Trained classifiers often showed racial bias, indicating that they were more likely to classify non-hateful AAE language as hate speech.

To prevent bias from affecting the results, a more balanced approach was adopted by reevaluating samples and lessening the relation between n-grams and certain labels. The new approach reduced bias very well. They used BERT-based transfer learning as their main model.
This paper is exceptionally relevant to my work because the data set I am working with contains a "target" category specifically outlining who the hate speech is directed toward. Across these expected target groups, my latter model might perform in one way or even another. For my milestone 2, I will be putting my Random Forest model to the test by testing to see if it treats hate speech fairly based on the amount of speech shared and how well it works for different target demographics. If I find bias, it could be beneficial to change my training data to include equal percentuage of all race and gender .This ethical issue is about more than just getting right answer.

Paper 6: Comparative Analysis of Hate Speech Detection: Traditional vs Deep

Learning (IEEE, 2024)

I found a conference paper in ieee that focused on comparison of both machine learning and deep learning for Hate speech. Researchers employed the Davidson dataset, containing 25,297 tweets rated as hateful offensive none. The authors tested: LinearSVC, Logistic Regression, Random Forest, and Naive Bayes, along with: LSTM, CNN, and BERT.
The results surprised me. Logistic regression was able to find the hate class a remarkable 55 percent of the time, while deep learning methods were unsuccessful in identifying it at nearly 49 percent. It has been seen that results of traditional models are actually better than the results of deep learning for hate speech detection due to training time and interpretability advantages as well. Both methods had a hard time telling the difference between hate and offensive speech.
This paper strongly validates my methodology choice. My Random Forest model that carefully selects key features (using TF-IDF, PCA, LDA, Chi-Square) will perform the most competitively deep learning models. Datasets with hate speech are imbalanced and few because the speech is rarely agreed upon. According to my HateXplain project of milestone 1, I realised that I had similar data imbalance, with about 26 percent offensive language, 16 percent hate speech, and 58 percent normal language. The paper reports that a train test split that uses weighted metrics and segregate are recommended.

Paper 7: A Survey on Automatic Online Hate Speech Detection in Low-Resource Languages (arXiv, 2024)

This extensive investigation looked at more than 50 studies on detecting hate speech in dozens of languages in different regions. Researchers investigated the most apporpriate features and studies for translating diverse languages. The best methods include TF-IDF, Word2Vec and different types of models(Random Forest and SVM). Deep neural network-based models(LSTM and BiLSTM) also work well on pattern based spotting tasks.

I have seen that their research indicated the "best" method depends upon the dataset and language, but that TF-IDF with Random Forest works well in many circumstances. This method appeals to me because many other researchers have tried it and seen that it is proven for work. They also addressed limited datasets, in low resource languages, but noted that they've gotten large datasets, in fact, to train the english hate speech detectors.

Traditional methods of machine learning may still be the most effective and efficient way to solve some problems. My choice to stick with Random Forest supports my focus to improve feature engineering before continuing on to more complex neural networks. The paper also discusses various feature selection techniques, which recalls my application of PCA, LDA, and Chi-Square.

Paper 8: Combating Hate Speech Using an Adaptive Ensemble Learning Model (PMC, 2021)

The focus of this paper was the methods to generalize to hate speech. The scientists understood how models frequently adjust to certain hate speech and regularly don't adapt to new hate speech patterns. An ensemble system was created by combining several algorithms(SVM, Random Forest and Logistic Regression) and weighted based on how efficiently they produce results.

Their approach was to train many classifiers which would be weighted properly afterwards. Researchers tested their method for detecting hate speech across a number of different datasets of hate speech. The adaptive ensemble produced better cross-dataset results than single models and fixed-weight ensembles.

Choosing Random Forest was relevant to my work because it uses many decision trees combined. Future research could utilize a variety of different machine compilation model types. I can try to use the Random Forest algorithm, but I need to process the data with PCA, LDA, and ChiSquare feature reduction to try to select the best data. It could result in more excellent performance than choosing just one feature engineering technique. Using different models is considerably stronger and has less of a chance to fit so poorly than a single reliability.

Paper 9: Leveraging CNN-BiLSTM for Multi-Class Cyberbullying Detection (IJMSRT, 2024)

Using a combination of CNNs and BiLSTMs to recognize bullying on the internet was proposed. Researchers in a similar study applied a deep learning-based method that uses computer vision algorithms called Convolutional Neural Networks and Bidirectional Long Short-Term Memory networks to identify similar patterns in tweets that may be considered cyberbullying.

The programmers used data augmentation, making the imbalance less of an issue by adjusting the parameters. Their model scored 91.07% accurately with classifying it. The text is also in many languages, including English and Hindi, which causes difficulty.

In comparing this to my work, simpler model complexity is the biggest variation sadly. A CNN-biLSTM fuses two networks to utilize a GPU. My Random Forest approach is much simpler and faster. However, I can learn from their data augmentation strategy. The data set also contains a problem with unbalanced class, with minority being hate speech. If I cant use a neural network I would use SMOTE to add balance in my classes. Fusing CNN and BiLSTM features conceptually resembles my proposed comparison of different engineering methods; both methods know that varied representations increase performance.

Paper 10: Exploring Unsupervised Feature Extraction Algorithms for High-Dimensional Data (Nature, 2025)

The paper I reviewed discussed various ways to limit the number of characteristics automatically contained in a dataset. It classifies the subjects PCA, Kernel PCA, LDA, ICA, NMF, Autoencoders, and t-SNE. The researchers tested the methods in many various large-scale datasets. The methods are being judged based on accuracy, speed, uniqueness, and how well they can create a copy of the original data.

Dimension reduction methods are specifically useful in the analysis of data when it has a certain type, non-linear patterns with noise, or in the classification of data. The single best method simply does not exist, yet combining several tactics secures the most excellence. PCA and LDA were infinitely quicker in comparison to autoencoders.
This paper directly informs my milestone 2 implementation. I would expect supervised LDA to outperform unsupervised PCA in classifying text as normal, offensive, or hateful due to LDA's focus on class distinction. Suddenly, PCA will be so important for new purposes. The Chi-Square technique aids in discovering phrases and words not typical to a certain group. Instead of using one method, I found that combining them made better results. I can create a combination of PCA, LDA, and Chi-Square to make more data available. This will provide the benefits of noise reduction from PCA, class separation from LDA, and interpretability for chi-Square.

Overall Synthesis and Comparison

After examining these 10 papers, multiple trends arise. There's no agreement on the top approach either. Traditional and deep learning methods both emerge as favourable. Successful results require carefully selected and analyzed data features consistently. Random sampling and use of correct algorithm are many of the advantages to computer programs making decisions over human ones. Using many methods all at once instead of just one method tend to work better.

I see a great deal of considerable evidence that a Random Forest approach to milestone 1 can be very useful. These papers confirm how well simple techniques for analyzing and classifying texts work compared to more difficult methods, known as complex approaches. The research backs up my idea, which involves multiple sets of methods, such as PCA feature selection, LDA, and Chi-Square performance comparison. I anticipate achieving a relatively lower accuracy rate of 80–85% using a simpler deep learning model that surpasses the highest results of existing models. I also won’t need powerful or expensive computers. Knowing every single feature that influences a model's decision is crucial. Understanding the language processes involved in situation evaluation is obviously important, verry helpful though.

Section 3:

Initial Data Preparation

Loading everything at once didn't seem a good strategy and I decided to select a percentage of the entire dataset. There was 24,449 normal posts,18,070 hate speech posts,and 17,925 offensive posts. To avoid any results that are misleading, data was divided into two sets: a 48,355 post set used to produce a prediction model and a 12,089 post set used to test the model to see whether it would return accurate predictions.
The very first thing I did was separating out the significant words. This program turned my social media posts into quantifiable computer data that machines could analyze. These features show you the

importance of certain words to the post.

Baseline Performance - Where I Started

The next step was to build a Random Forest model that will utilize all of the TF-IDF features that were extracted. At first, I wanted to see a base number as well. The model worked at a 64.94 percent efficiency. This baseline helped me figure out if my feature engineering would really help the outcome.

I looked at the baseline and found something pretty cool. My model for comment rating did best with normal comments (72% precision, 76% recall), performing at a higher percentage than comments classed as "hateful" or "offensive". My model early did better on comments rated as not good than comments classed as hateful or offensive and did worst with comments classed as "offensive"(only 52% precision, 48% recall). The distinction between harmful comments that are offensive and hateful isn't apparent - there are similarities between the two.

Code:

```
# Create TF-IDF features from post_tokens
tfidf_vectorizer = TfidfVectorizer(
    max_features=1000,
    min_df=2,
    max_df=0.8,
    ngram_range=(1, 2),
    stop_words='english'
)

# Transform text to TF-IDF features
X_tfidf = tfidf_vectorizer.fit_transform(df_clean['post_tokens'])
X_tfidf_dense = X_tfidf.toarray()



# Train Random Forest classifier
rf_clf_base = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
print(f"\nTraining Random Forest with {X_train_base.shape[1]} features...")
rf_clf_base.fit(X_train_base, y_train_base)

# Predictions
y_pred_base = rf_clf_base.predict(X_test_base)
```

Method 1: PCA - Less Effective Than I Expected

Selecting PCA was my desired starting point for reducing the features of my model and eliminating minimal loss of performance. Each experiment contained a different number of components, fifty, hundred, and two hundred.

I'm concerned that PCA's only capture of 26.11% data resolution might damage my model, because of fifty different components. In reality, yes, the accuracy went down 64.48 percent - just a 0.45 percent decline. Despite a few mishaps, the model turned out to work fairly well.

When I increased to 100 components, I captured 37.37% of variance and I achieved accuracy of only 64.22%, losing 0.72%. Initial surprise quickly turned into an understanding that extraneous data can instead hurt, not help. My model's performance can be hindered by noise if it disrupts something.

Limiting PCA to only 200 units reduced the accuracy rate to 64 percent and captured 51.85 percent of the total variance. The results kept sliding down as I added more components. The next best configuration was 50 components at 64.48%. Data analysis indicated to me that PCA wasn't effective. The feature reduction wasn't helpful with the minor accuracy loss.

Code:

```
# Initialize PCA
pca = PCA(n_components=n_comp, random_state=42)

# Transform features
X_pca = pca.fit_transform(X_tfidf_dense)
```

Method 2: LDA - The Winner!

Then, I attempted Linear Discriminant Analysis. Unlike PCA, LDA aims to categorize normal from offensive and ultimately from hate speech. Because I am taking 3 classes, LDAs software only accepts 2 components - that is out of my control due to mathematical and time reasons.

This's where events got compelling. I was able to get 64.86 percent by having 2 parts aside from 1000 features. By eliminating 99.8% of the features in my data, I was able to find a perfect separation between classes using LDA.

In looking at those findings the most significant and relevant trends would ideally vary between just a few categories. The model discovered these routes by itself.

Code:

```
# Apply LDA
```

```
lda = LDA(n_components=max_lda_components)
X_lda = lda.fit_transform(X_tfidf_dense, y)
```

Method 3: Chi-Square - Great for Interpretability

I found out what terms and phrases were the best at separating two groups. I used 50 words and got 61.97% (2.97 percent drop). I discovered these insulting words connected to hate speech as hate speech markers.

Increasing the value to 100 gave me 63.23 percentage accuracy, a drop of 1.7 percent. After setting k=200, my accuracy actually improved to 64 .07 % while actually lowering the difference to 0.87 % lower . With half the amount of features I had, I managed to still get 64.65 percent.

One thing I enjoyed about Chi-Square was that I saw precisely what attributes had been chosen. The top themes with k=500 included words like "africa" and "african", typical words that pop up when you talk about rage. I see this interpretability as the enlightenment of what exactly my model is using.

Code:

```
# Initialize Chi-Square selector
chi2_selector = SelectKBest(chi2, k=k)
 # Transform features
X_chi2 = chi2_selector.fit_transform(X_tfidf_dense, y)
```

Method	Features	Accuracy	Change
Baseline	1000	64.94%	-
PCA (50)	50	64.48%	-0.45%
LDA (2)	2	64.86%	-0.07%
Chi-Square (500)	500	64.65%	-0.29%

The only baseline with all 1,000 features certainly showed the best overall. One thing that really surprised me however came from using LDA, which only came to 64.86%, but achieved a result close to actual percentages. If I put more emphasis on being quicker, I would use LDA due to its smaller size.

Lessons I Learned From This

I initially thought that reducing the features would make a difference. I supposed that removing noise would enhance my model. I found out that my hate speech dataset which consists of all nearly a thousand features shows all information that I was trying to discover. Reducing dimensions leads to

lesser predictive outcome.

However, LDA showed me something valuable. Getting rid of one thousand pieces of information shows a 99.93 percent improvement in accuracy, implying there is unnecessary data being used. LDA found the most important directions naturally.

When the k value was 500, a Chi-Square analysis showed the importance of the words and selectiveness of keeping them was more than enough, with almost minimal loss, nearly always.

Real-World Trade-offs

If I were bringing this model to production, my choice would depend on my limitations.

- Use LDA with two components for speed-critical tasks that can tolerate small losses in accuracy.
- Use Chi-Square (when k=200 or more) if I need to justify my choices.
- When you only care about accuracy: Stick with baseline (all the 1,000 features).

As part of my project however, I have shown that I know how to utilize and contrast three different feature engineering methods. I revealed their trade-offs clearly. Just because the baseline did best doesn't imply my analysis was a shortcoming. It essentially shows that my features were already great and that my extraction out of TF-IDFs with bigrams and stopword removal was good.
I can now use some solid numbers to illustrate in my report on the relevance of the right feature engineering choice on the model performance.